

TEST PLAN

Traffic Management System (Simulation)

Software Engineering — Mini Project

IEEE-Style Test Plan and Test Case Specification

Document	Test Plan + Test Case Update
Project	Traffic Management System (Simulation)
Implementation	C/C++ standalone console simulation
Primary data structure	Queues
Test cases	12
Source documents	Software Requirements Specification (SRS) and Software Architecture & Design Specification (SADS)

1. Introduction

1.1 Purpose

This Test Plan defines the approach, scope, environment, and organization for testing the Traffic Management System (Simulation). Its purpose is to verify that the implemented system satisfies the Functional Requirements (FR1–FR8), Non-Functional Requirements (NFR1–NFR3), Security Objectives (SO1–SO2), and Security Requirements (SR1–SR2) defined in the project's Software Requirements Specification (SRS), and that the behavior of the system is consistent with the Software Architecture & Design Specification (SADS).

This document covers only the Test Plan and the associated Test Case Update deliverables. It does not redefine, redesign, or modify any requirement, component, or architectural decision described in the SRS or SADS.

1.2 Scope

The scope of testing covers the standalone C/C++ Traffic Management System (Simulation) described in the SRS (§1.2) and SADS (§A.1). Testing addresses:

- All functional requirements FR1–FR8 (adding vehicles, maintaining queues, signal transitions and states, vehicle entry/exit, displaying traffic information, congestion detection, and traffic statistics).
- All non-functional requirements NFR1 (Performance), NFR2 (Usability), and NFR3 (Reliability).
- Security objectives SO1 (Data Integrity) and SO2 (Input Validation), and security requirements SR1 (Input Validation) and SR2 (Queue Data Protection).
- Integration behavior between the layered components identified in the SADS: Simulation UI, Simulation Controller, Vehicle Manager, Signal Controller, Congestion Detector, Statistics Manager, and Queue Manager.

Out of scope: networking, authentication, encryption, multi-user, or persistence testing, because the SRS defines the system as a standalone, single-user, offline simulation with no network exposure.

1.3 References

- Software Requirements Specification (SRS) — Traffic Management System (Simulation).
- Software Architecture & Design Specification (SADS) — Traffic Management System (Simulation).
- IEEE 829 Standard for Software and System Test Documentation (format reference).

1.4 Definitions and Acronyms

Term / Acronym	Meaning
FR	Functional Requirement, as defined in SRS §3.1 (FR1–FR8).
NFR	Non-Functional Requirement, as defined in SRS §3.2 (NFR1–NFR3).
SO	Security Objective, as defined in SRS §5.1 (SO1–SO2).
SR	Security Requirement, as defined in SRS §5.2 (SR1–SR2).
Vehicle	A simulated vehicle in the system.
Queue	A data structure used to maintain waiting vehicles.
Traffic Signal	Red, Yellow, or Green signal controlling vehicle movement.
Queue Length	Number of vehicles currently waiting.
Congestion	A lane having more vehicles than the defined threshold.
TC	Test Case.

1.5 Test Objectives

- Verify that every functional requirement (FR1–FR8) behaves as specified in the SRS.
- Verify that the non-functional qualities (performance, usability, reliability) required by NFR1–NFR3 are observable in the running simulation.
- Verify that SO1, SO2, SR1, and SR2 are enforced, specifically that invalid input is rejected before it reaches the Vehicle Manager or Queue Manager, and that queue state cannot be corrupted, lost, or duplicated.

- Establish traceability between each requirement and at least one test case.
- Identify defects prior to submission of the mini-project deliverable.

2. Test Strategy / Test Approach

2.1 What Will Be Tested

Testing is organized around the system's layered architecture: Presentation Layer (Simulation UI), Simulation/Control Layer (Simulation Controller), Traffic Management Layer (Vehicle Manager, Signal Controller, Congestion Detector, Statistics Manager), and Data Structure Layer (Queue Manager). Each requirement is exercised through the interfaces exposed by these components.

2.2 Functional Testing

Functional testing verifies FR1–FR8 through the system's defined operations: `addVehicle`, `removeVehicle`, `getQueueLength`, `updateSignal`, `getSignalState`, `detectCongestion`, `calculateStatistics`, and `displayTrafficInfo`. Outputs are compared with the behavior defined in SRS §3.1 and the component interactions described in the SADS.

2.3 Non-Functional Testing

- NFR1 (Performance): exercise a representative volume of vehicles and queue operations and confirm successful completion without crash, hang, freeze, or failure. The SRS does not define a numerical response-time threshold.
- NFR2 (Usability): check that the Simulation UI output contains the required signal states, queue information, and vehicle counts in a consistent and understandable form.
- NFR3 (Reliability): check queue state before and after valid and invalid operations to confirm that vehicles are not lost or duplicated.

2.4 Security Validation

Security validation is treated as a dedicated activity in Section 5.1. It confirms that invalid vehicle/lane input is rejected before it reaches the Vehicle Manager and that the Queue Manager refuses operations that could corrupt queue state.

2.5 Integration / System Testing

Integration testing follows the two sequence flows defined in the SADS: the Add Vehicle flow and the Vehicle Movement/Signal Change flow. System-level test cases exercise these end-to-end paths rather than only isolated components.

2.6 Test Levels and Techniques

- Unit-level checks on individual component interfaces where practical, such as Queue Manager `enqueue/dequeue` behavior.
- Integration testing across the Simulation Controller and the components it coordinates.
- System testing of complete scenarios such as vehicle addition, signal cycles, congestion detection, and statistics reporting.
- Boundary and negative testing for invalid inputs and invalid queue operations (SR1, SR2).

3. Test Environment

3.1 Development and Execution Environment

- Language/Platform: C/C++ standalone console application.
- Execution mode: single-process, single-thread simulation with no network exposure.
- Operating environment: a workstation capable of compiling and running a standard C/C++ console program.

3.2 Required Software / Tools

- A C/C++ compiler such as GCC/G++ or an equivalent toolchain.
- A console/terminal to run the compiled simulation and observe displayed output.
- A text editor or IDE for reviewing source code and test scripts, where applicable.
- A test-case tracking sheet to record actual results and Pass/Fail status.

3.3 Input Conditions

- Add-vehicle commands specifying a lane identifier and vehicle identifier.
- Signal cycle ticks that advance each lane's Red/Yellow/Green state.
- Read/query operations for queue length, signal state, congestion status, or statistics.
- Deliberately invalid inputs, such as malformed vehicle/lane identifiers or operations on an empty queue, used only for negative and security testing.

3.4 Assumptions

- Vehicles follow the predefined traffic rules.
- Signals operate according to a defined cycle.
- Congestion is determined using a predefined queue-length threshold.
- No additional assumptions such as network availability, multi-user access, or persistence are made.

4. Test Organization / Test Procedures

4.1 Testing Responsibilities

The tester is responsible for designing test cases, executing them against the compiled simulation, recording actual results, and reporting defects. Development of the system components and SADS architecture/design documentation remains the responsibility of the corresponding teammates.

4.2 Test Execution Approach

Execute tests in feature order: Queue Manager and Vehicle Manager operations (FR1, FR2, FR5); Signal Controller behavior (FR3, FR4); Congestion Detector and Statistics Manager behavior (FR7, FR8); Simulation UI display (FR6); and finally non-functional and security validation tests.

4.3 Entry Criteria

- Relevant source code has been compiled successfully without errors.
- The SRS requirements and corresponding SADS component descriptions are available and unchanged.
- Test cases and expected results have been reviewed and are ready for execution.

4.4 Exit Criteria

- All test cases listed in Section 5 have been executed at least once.
- Every FR1–FR8, NFR1–NFR3, and SR1–SR2 is traceable to at least one test case through the SRS Traceability Matrix in Section 6.
- All high-severity defects identified during testing have been resolved and re-tested, or explicitly deferred with justification.

4.5 Test Procedure

- Prepare: build the simulation executable and confirm it starts without error.
- Execute: run each test case's input/steps against the compiled simulation.
- Observe: record the actual output shown by the Simulation UI or returned by the relevant function interface.
- Compare: compare actual results against the expected result defined for the test case.
- Record: mark the test case Pass or Fail and note remarks.
- Report: log any Fail result as a defect using the Test Case ID and Requirement ID.

4.6 Defect / Error Handling

When a test reveals unsafe behavior, such as a vehicle being lost or duplicated or invalid input being accepted, the defect is logged with the Test Case ID, Requirement ID, steps, expected result, and actual result. The defect is classified, reported to the responsible developer, and the associated test case is re-executed after a fix. Defects involving SR1 or SR2 receive high priority because they underpin data integrity and input validation.

5. Test Cases and Expected Results

The following twelve test cases cover all functional requirements (FR1–FR8), all non-functional requirements (NFR1–NFR3), and the security requirements (SR1, SR2). The Actual Result and Pass/Fail fields are intentionally left blank for completion during test execution.

Test Case Records — Part A

TC-01 — FR1, SR1

Requirement ID(s)	FR1, SR1
Test Objective	Verify that a vehicle can be added to a valid lane with valid inputs.
Preconditions	Simulation is running; lane identifiers are initialized and empty.
Test Input / Steps	1. Issue addVehicle(laneId, vehicleId) with a valid lane ID and vehicle ID. 2. Observe confirmation output.
Expected Result	Input Validator accepts the input; Vehicle Manager creates the vehicle and enqueues it via Queue Manager; the UI displays success confirmation.
Actual Result	
Pass / Fail	
Remarks	Covers the add-vehicle path defined in the SADS.

TC-02 — FR2, FR5

Requirement ID(s)	FR2, FR5
Test Objective	Verify that vehicles are maintained in per-lane FIFO queues and can enter/leave correctly.
Preconditions	At least one lane has zero or more vehicles queued.
Test Input / Steps	1. Add V1, V2, V3 to the same lane in order. 2. Query getQueueLength(laneId). 3. Trigger a move/removeVehicle(laneId) operation.
Expected Result	Vehicles remain in FIFO order; queue length is 3 before removal; V1 is dequeued first; queue length becomes 2.
Actual Result	
Pass / Fail	
Remarks	Confirms Queue Manager FIFO behavior.

5. Test Cases and Expected Results (continued)

Test Case Records — Part B

TC-03 — FR3, FR4

Requirement ID(s)	FR3, FR4
Test Objective	Verify that traffic signals transition through Red, Yellow, and Green according to the defined cycle.
Preconditions	Lane Signal Controller is initialized to a known starting state.
Test Input / Steps	1. Call <code>updateSignal()</code> repeatedly. 2. Query <code>getSignalState(laneId)</code> after each tick.
Expected Result	Signal state advances only through Red, Yellow, and Green in the defined cycle; no other signal state is observed.
Actual Result	
Pass / Fail	
Remarks	Validates Signal Controller behavior.

TC-04 — FR5, FR3

Requirement ID(s)	FR5, FR3
Test Objective	Verify that a vehicle exits the queue only once its lane's signal reaches Green.
Preconditions	A lane has at least one queued vehicle; signal is Red or Yellow.
Test Input / Steps	1. Add a vehicle while signal is Red/Yellow. 2. Observe queue during Red/Yellow. 3. Advance the normal cycle to Green and observe again.
Expected Result	Vehicle remains queued during Red/Yellow; when signal reaches Green, the Simulation Controller directs the Vehicle Manager to dequeue and move the front vehicle.
Actual Result	
Pass / Fail	
Remarks	Black-box observation of normal run-loop behavior; no direct off-cycle dequeue is assumed.

5. Test Cases and Expected Results (continued)

Test Case Records — Part C

TC-05 — FR6, NFR2

Requirement ID(s)	FR6, NFR2
Test Objective	Verify that the Simulation UI clearly displays current signal state, queue contents, and vehicle counts.
Preconditions	At least one lane has a non-empty queue and known signal state.
Test Input / Steps	1. Trigger <code>displayTrafficInfo()</code> . 2. Compare console output against a fixed checklist.
Expected Result	For every lane, output includes lane identifier, current Red/Yellow/Green state, and number of vehicles waiting; values match simulation state.
Actual Result	
Pass / Fail	
Remarks	Checklist-based criterion makes the usability check objective.

TC-06 — FR7

Requirement ID(s)	FR7
Test Objective	Verify that a lane is reported as congested when its queue length exceeds the defined threshold.
Preconditions	Congestion threshold is configured; lane starts below threshold.
Test Input / Steps	1. Add vehicles until queue length exceeds threshold. 2. Call <code>detectCongestion(laneId)</code> .
Expected Result	Lane is reported congested once queue length exceeds the threshold. Also verify exactly-at-threshold is not congested and one-above-threshold is congested.
Actual Result	
Pass / Fail	
Remarks	Boundary case for FR7.

5. Test Cases and Expected Results (continued)

Test Case Records — Part D

TC-07 — FR7

Requirement ID(s)	FR7
Test Objective	Verify that a lane below the congestion threshold is reported as not congested.
Preconditions	Lane queue length is below the configured threshold.
Test Input / Steps	1. Add a small number of vehicles while staying below threshold. 2. Call detectCongestion(laneId).
Expected Result	detectCongestion returns not-congested status.
Actual Result	
Pass / Fail	
Remarks	Negative case for FR7.

TC-08 — FR8

Requirement ID(s)	FR8
Test Objective	Verify that traffic statistics are calculated correctly.
Preconditions	Vehicles have been added to and removed from queues during the run.
Test Input / Steps	1. Add and move several vehicles. 2. Call calculateStatistics().
Expected Result	Statistics include waiting time, queue length, and vehicles processed, consistent with the recorded vehicle-added and vehicle-departed events.
Actual Result	
Pass / Fail	
Remarks	Cross-check processed count against move operations.

5. Test Cases and Expected Results (continued)

Test Case Records — Part E

TC-09 — NFR1

Requirement ID(s)	NFR1
Test Objective	Verify that the system handles a representative volume of vehicle and queue operations efficiently.
Preconditions	Simulation is running and ready to accept commands.
Test Input / Steps	1. Add a representative larger volume of vehicles across multiple lanes. 2. Trigger signal cycles, moves, and statistics calls. 3. Observe whether the program completes operations and remains responsive.
Expected Result	All operations complete successfully for the selected test volume without crash, hang, freeze, or failure. The console remains responsive throughout. No numerical response-time limit is imposed because the SRS does not define one.
Actual Result	
Pass / Fail	
Remarks	Objective observable criterion; no invented timing threshold.

TC-10 — NFR3, SR2

Requirement ID(s)	NFR3, SR2
Test Objective	Verify that queue state cannot be lost, duplicated, or corrupted, including under an invalid operation.
Preconditions	A lane's queue is empty.
Test Input / Steps	1. Attempt removeVehicle(laneId) / dequeue on the empty queue. 2. Add vehicles and repeat valid enqueue/dequeue operations. 3. Verify queue contents after each step.
Expected Result	Empty-queue operation is rejected/guarded without corrupting state; subsequent valid operations leave exactly the expected vehicles in the expected order, with none lost or duplicated.
Actual Result	
Pass / Fail	
Remarks	Directly exercises SR2 queue protection.

5. Test Cases and Expected Results (continued)

Test Case Records — Part F

TC-11 — SR1, SO2

Requirement ID(s)	SR1, SO2
Test Objective	Verify that invalid vehicle/lane input is rejected before it reaches the Vehicle Manager or Queue Manager.
Preconditions	Simulation is running.
Test Input / Steps	1. Submit an invalid/malformed lane ID or vehicle ID. 2. Observe response and queue state.
Expected Result	Input Validation rejects the request and displays an error; no vehicle is created and no queue is modified.
Actual Result	
Pass / Fail	
Remarks	Confirms the invalid-input path in the SADS.

TC-12 — SO1

Requirement ID(s)	SO1
Test Objective	Verify that vehicle, queue, signal, and statistics data remain consistent across a mixed sequence of valid and invalid operations.
Preconditions	Simulation is running with at least one lane containing vehicles.
Test Input / Steps	1. Interleave valid add/move operations with invalid inputs and invalid queue operations. 2. Compare final state with the expected state derived only from valid operations.
Expected Result	Invalid input and invalid queue operations have no effect on simulation state; final vehicle, queue, signal, and statistics data remain consistent with valid operations only.
Actual Result	
Pass / Fail	
Remarks	Composes SR1 and SR2 to demonstrate SO1.

5.1 Security Validation

This subsection presents dedicated security validation for the security objectives SO1 (Data Integrity) and SO2 (Input Validation) and security requirements SR1 (Input Validation) and SR2 (Queue Data Protection).

Security ID	Requirement	Test Case	Security Test Description	Expected Result
SR1	The system shall validate vehicle and simulation inputs before adding or processing them.	TC-11	Submit malformed/invalid lane and vehicle identifiers and confirm they are intercepted by the Input Validation step before the Vehicle Manager is invoked.	Invalid input is rejected; an error is shown; no vehicle or queue state is created or modified.
SO2	The system shall ensure that user inputs are validated before being processed to prevent invalid data from affecting the simulation.	TC-11	Confirm that no invalid data reaches the Vehicle Manager or Queue Manager as a result of the SR1 validation gate.	Simulation state is unaffected by rejected input.
SR2	The system shall prevent invalid operations from causing unintended loss, duplication, or corruption of vehicles in traffic queues.	TC-10	Attempt an invalid queue operation such as dequeue on an empty queue and confirm the Queue Manager rejects it without corrupting queue contents.	Operation is refused/guarded; queue contents and order remain correct; no vehicle is lost or duplicated.
SO1	The system shall maintain the integrity of vehicle, queue, signal, and traffic-statistics data throughout the simulation.	TC-12	Interleave valid operations with invalid input and invalid queue operations, then compare final state with the expected state from valid operations only.	Data integrity is preserved because invalid input and invalid queue operations do not modify simulation state.

6. SRS Traceability Matrix

The matrix maps every requirement defined in the SRS to the test case(s) that verify it.

Requirement ID	Requirement	Test Case ID(s)	Expected Verification
FR1	The system shall allow users to add vehicles to the traffic simulation.	TC-01, TC-11	Vehicle is created and enqueued for valid input; invalid input is rejected.
FR2	The system shall maintain vehicles in queues based on traffic lanes and traffic signal states.	TC-02	Vehicles remain in correct per-lane FIFO order; queue length is reported accurately.
FR3	The system shall transition traffic signals between Red, Yellow, and Green states according to a defined signal cycle.	TC-03, TC-04	Signal advances only through the defined Red→Yellow→Green cycle.
FR4	The system shall implement traffic signal states such as Red, Yellow, and Green.	TC-03	getSignalState returns only Red, Yellow, or Green.
FR5	The system shall allow vehicles to enter and leave queues according to defined traffic rules.	TC-02, TC-04	Vehicles enter on valid add and leave only on Green, in FIFO order.
FR6	The system shall display the current traffic signal state, vehicle queues, and the number of vehicles waiting in each lane.	TC-05	Displayed output matches actual signal state and queue lengths.
FR7	The system shall identify lanes experiencing congestion based on the number of vehicles waiting in their queues.	TC-06, TC-07	Congestion is reported above threshold and not reported below threshold.
FR8	The system shall calculate basic traffic statistics such as waiting time, queue length, and number of vehicles processed.	TC-08	Reported statistics match the actual sequence of add/move operations.
NFR1	The system should process vehicle movements and queue operations efficiently.	TC-09	Representative workload completes without crash, hang, freeze, or failure; console remains responsive.
NFR2	The system should provide clear and understandable output showing traffic signals, vehicle queues, and traffic statistics.	TC-05	Required output elements are present, correctly labeled, and match simulation state.
NFR3	The system should correctly maintain vehicle queues without losing or duplicating vehicles.	TC-02, TC-10	Queue contents and order remain correct after valid and invalid operations.
SR1	The system shall validate vehicle and simulation inputs before adding or processing them.	TC-11	Invalid input is rejected before Vehicle Manager processing.
SR2	The system shall prevent invalid operations from causing unintended loss, duplication, or corruption of vehicles in traffic queues.	TC-10	Invalid queue operation is refused and queue integrity is preserved.

7. Test Case Summary

Category	Coverage / Count
Total test cases	12
Functional requirements	8 primary functional cases (TC-01 through TC-08), covering FR1–FR8.
Non-functional requirements	All 3 covered: NFR1 by TC-09, NFR2 by TC-05, and NFR3 by TC-02 and TC-10.
Security validation	SR1/SO2 validated by TC-11; SR2 validated by TC-10; SO1 validated by TC-12.
Requirement traceability	FR1–FR8, NFR1–NFR3, SO1–SO2, and SR1–SR2 are mapped to test cases in Section 6.

Submission checklist:

- Test Plan includes introduction and descriptive sections.
- Sections 3, 4, and 5 are completed.
- Section 5.1 provides dedicated security validation.
- At least 7–10 test cases are required; this document contains 12.
- Functional and non-functional requirements are covered.
- SRS traceability is included in Section 6.
- Actual Result and Pass/Fail fields remain blank for execution and should be completed when testing is performed.

End of Test Plan and Test Case Update.